

CCE1 EXAMINATION QUESTION BANK — MASTER CONSOLIDATED SOLUTIONS

Course Pattern: TE IT - 2024 Pattern (SPPU)

Academic Term: Semester V (2025-26 / 2026-27)

Document Type: Master Consolidated Solutions (Reundancy Grouped) **Course Code:** PCC-301-ITT / PCC-301-IT

IMPORTANT:

Consolidated Question Bank Architecture: This master document synthesizes and groups all redundant and overlapping questions from the official CCE1 Question Bank (covering all 28 questions across Unit 1 and Unit 2) into **15 Comprehensive Topic Solutions**. Every section explicitly maps the original Question Bank IDs, Bloom's Taxonomy Levels (BTL), and marks weightage. Full mathematical Gantt charts and step-by-step numerical calculations are provided for all scheduling problems.

Unit 1: Overview of Operating System & System Structures (CO301.1)

Topic 1.1: Operating System Definition, Basic Functions, Objectives & Services

GROUP:

Consolidated Questions Covered:

- **Q1:** Define Operating System. List its basic functions.
- **Q2:** What are the objectives of an Operating System?
- **Q5:** State and explain different services provided by an operating system.

1. Definition of an Operating System: An **Operating System (OS)** is system software that acts as an intermediary between computer hardware and the computer user. It manages computer hardware resources (CPU, Memory, Storage, I/O devices) and provides a convenient, secure environment for application programs to execute efficiently.

2. Primary Objectives of an Operating System:

1. **Convenience:** Hides complex hardware architecture and low-level machine instructions from end users and programmers, presenting a user-friendly abstraction.
2. **Efficiency:** Ensures optimal utilization of hardware resources (CPU scheduling, memory allocation, storage distribution) among competing processes.
3. **Ability to Evolve:** Structured modularly to permit effective development, testing, and introduction of new system functions without interfering with service.

3. Basic Functions & Core System Services:

- **Process Management:** Creation, scheduling, synchronization, and termination of processes and threads.
- **Main Memory Management:** Dynamic tracking of memory bytes, allocation/deallocation of memory blocks, and virtual memory paging/swapping.
- **File System Management:** Creation, deletion, directory organization, and mapping of logical files to physical secondary storage blocks.
- **I/O System Management:** Buffering, caching, spooling, and providing device-driver interfaces to abstract specific hardware devices.

- **Protection and Security:** Regulating access to system resources via dual-mode CPU protection and user authentication.
- **Error Detection & Handling:** Continuous monitoring for hardware glitches (memory parity errors, disk faults) and software exceptions (division by zero, illegal memory access).

Source: Silberschatz et al., Ch 1 & 2, p. 3-65

Topic 1.2: Dual-Mode Operation: User Level vs. Kernel Level Execution

GROUP:

Consolidated Questions Covered:

- **Q4:** Differentiate between user level and kernel level.

1. Concept of Dual-Mode Operation: To prevent errant or malicious user programs from crashing the operating system or modifying system data structures, modern CPU hardware enforces at least two independent modes of execution governed by a hardware **Mode Bit** (0 for Kernel Mode, 1 for User Mode).

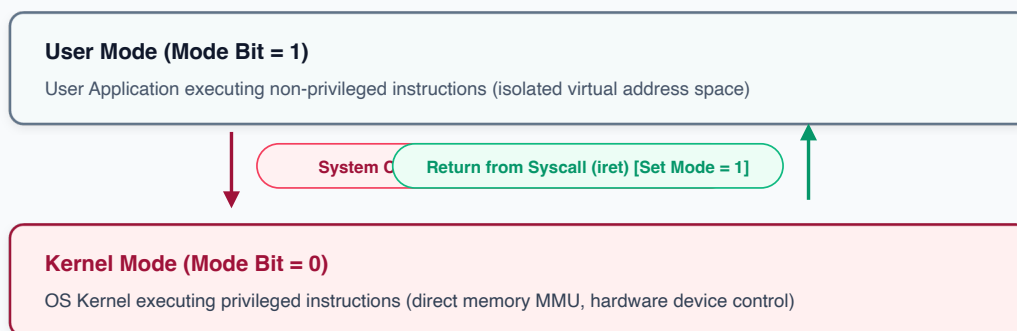


Figure 1.1: Dual-Mode Operation and System Call Transition

2. Parameter-Based Comparison: User Level vs. Kernel Level:

Parameter	User Level (User Mode)	Kernel Level (Kernel / Supervisor Mode)
Hardware Mode Bit	Mode Bit = 1	Mode Bit = 0
Privilege Level	Restricted execution; non-privileged instructions only	Unrestricted access to CPU instructions, MMU, and memory
Direct Hardware Access	Prohibited; cannot directly access I/O ports or physical registers	Direct control over hardware controllers, disk DMA, and CPU timers
Memory Protection	Isolated within user-space virtual address space	Full access to physical RAM and kernel address space
Fault Impact	Process crash / segmentation fault isolated to the application	Fatal kernel panic / system-wide OS crash / Blue Screen
Transition Mechanism	Transitions to Kernel Mode via Trap, System Call, or Hardware Interrupt	Returns to User Mode via sysret / iret instruction

Source: Silberschatz et al., Ch 1, p. 20-28

Topic 1.3: Shell Scripting & Practical Utilities

GROUP:

Consolidated Questions Covered:

- **Q3:** Write a BASH shell script to find factorial of a number and explain constructs.
- **Q6:** Explain shell commands with examples: grep, sort, chmod, cat, echo.
- **Q11:** Write a shell script to check file or directory existence.

1. Factorial BASH Script with Shell Construct Analysis:

```
#!/bin/bash
# Factorial calculation script in BASH
echo -n "Enter a positive integer: "
read num

if [ -z "$num" ] || [ "$num" -lt 0 ]; then
    echo "Error: Please enter a valid non-negative integer."
    exit 1
fi

fact=1
temp=$num

while [ $temp -gt 1 ]; do
    fact=$((fact * temp))
    temp=$((temp - 1))
done

echo "The factorial of $num is: $fact"
```

• Constructs Explained:

- read num: Reads user input from standard input into variable num.
- if [...]; then ... fi: Conditional branching construct testing input validation.
- while [\$temp -gt 1]; do ... done: Iterative loop construct executing multiplication.
- \$((expression)): Built-in BASH arithmetic expansion.

2. Essential Shell Commands Explained with Examples:

- grep (Global Regular Expression Print): Searches text patterns within files.
 - Example: grep "error" /var/log/syslog (Finds all lines containing "error").
- sort: Orders lines of text alphabetically or numerically.
 - Example: sort -n numbers.txt (Sorts lines numerically).
- chmod (Change Mode): Modifies filesystem access permissions.
 - Example: chmod 755 script.sh (Owner: rwx, Group/Others: r-x).
- cat (Concatenate): Reads and outputs file contents to terminal.
 - Example: cat file1.txt file2.txt > combined.txt.
- echo: Prints text strings or variable values to stdout.
 - Example: echo "Current Path: \$PATH".

3. File / Directory Existence Check Script:

```
#!/bin/bash
echo -n "Enter path to check: "
read target_path

if [ -d "$target_path" ]; then
    echo "'$target_path' is a valid Directory."
elif [ -f "$target_path" ]; then
    echo "'$target_path' is a regular File."
    ls -lh "$target_path"
else
    echo "Path '$target_path' does not exist."
fi
```

Source: Advanced BASH Scripting Guide, Ch 4-7

Topic 1.4: Operating System Structure & Design (Monolithic, Microkernel, Layered, Modular)

GROUP:

Consolidated Questions Covered:

- **Q9:** Analyze monolithic and microkernel architectures with respect to structure and performance.
- **Q10:** Compare monolithic and microkernel architectures.
- **Q12:** Discuss layered and modular approach to OS design.

1. Monolithic vs. Microkernel Architectures:

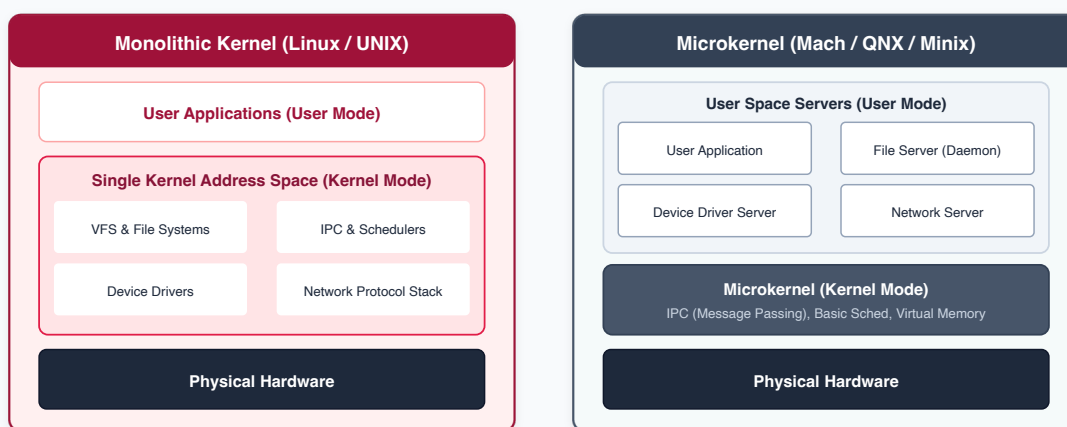


Figure 1.2: Monolithic Kernel vs. Microkernel Architecture

2. Parameter-Based Comparison: Monolithic vs. Microkernel:

Parameter	Monolithic Kernel	Microkernel
Structure	All OS services (FS, IPC, drivers, networking) reside in a single large kernel space address space	Minimal core (IPC, basic scheduling, virtual memory) in kernel space; services run as user-space servers
Performance	High throughput; services communicate via direct internal function calls with zero IPC overhead	Lower throughput; communication requires frequent message passing (IPC) and user-to-kernel mode switches

Parameter	Monolithic Kernel	Microkernel
Fault Isolation	Poor; a bug or memory leak in a single device driver crashes the entire operating system	Excellent; a crashing file system or driver server is isolated and restarted in user space
Extensibility	Difficult; modifying kernel services requires re-compiling the kernel image or dynamic module insertion	Easy; new services are added simply by starting new user-space daemon servers
Representative Examples	Linux, FreeBSD, MS-DOS, Traditional UNIX	Mach, QNX, Minix 3, seL4

3. Layered and Modular Approaches:

- **Layered Approach (Dijkstra's THE System):** OS is divided into N hierarchical layers (0 to N), where Layer 0 is Hardware and Layer N is User Interface. Each layer is implemented solely using operations of lower layers. Ensures modular verification but suffers from performance overhead and strict layering difficulties.
- **Modular Approach (Loadable Kernel Modules - LKM):** Core kernel provides essential functions; dynamic modules (device drivers, file system modules) are loaded into kernel space on-demand during runtime without rebooting (used in modern Linux and macOS). Combines monolithic performance with modular flexibility.

Source: Silberschatz et al., Ch 2, p. 70-85

Topic 1.5: Symmetric Multiprocessing (SMP), Virtual Machines & OS Performance Evaluation

GROUP:

Consolidated Questions Covered:

- **Q7:** Explain the concept of virtual machine with its benefits.
- **Q8:** Demonstrate application of SMP; illustrate concurrent execution and benefits over uniprocessors.
- **Q13:** Evaluate different operating systems based on performance.

1. Symmetric Multiprocessing (SMP):

- **Architecture:** Multiple identical physical CPU cores share a single common physical RAM bus and I/O system, all controlled by a single operating system instance.
- **Concurrent Execution:** All processors can execute independent processes or threads simultaneously without a master-slave bottleneck.
- **Benefits over Uniprocessors:**
 - **Increased Throughput:** More processes completed per unit time.
 - **Economy of Scale:** Processors share power supplies, memory, and motherboard buses.
 - **Enhanced Reliability / Graceful Degradation:** If one CPU core fails, the system continues operating at reduced capacity (fault tolerance).

2. Virtual Machine Concepts & Benefits: A **Virtual Machine (VM)** is a software abstraction of a complete computer system created by a Hypervisor (VMM).

- **Benefits:** Hardware consolidation (running Linux and Windows on one server), secure sandboxing, snapshotting/rollback, and instant environment provisioning.

3. Performance Evaluation Criteria across Operating Systems:

- **CPU Utilization & Throughput:** Percentage of active CPU compute cycles vs. context switch overhead.
- **Response Time & Latency:** Millisecond turnaround for user I/O events (critical in Real-Time OS like QNX vs. General Purpose Linux/Windows).
- **Scalability & Memory Footprint:** Efficiency in scaling from 4 to 256 CPU cores under heavy multi-threaded database workloads.

Source: Stallings, Ch 1 & 2, p. 45-78

Topic 1.6: Linux Booting Process

GROUP:

Consolidated Questions Covered:

- **Q14:** Explain Linux booting process in detail.

Step-by-Step Linux Boot Sequence:



End-to-End Boot Pipeline from Power Switch to Interactive Terminal

Figure 1.3: Chronological Linux Boot Sequence

1. **BIOS / UEFI Phase:** Executes Power-On Self-Test (POST) to verify hardware integrity and reads the boot device order from CMOS RAM.
2. **Bootloader Phase (GRUB2):** Loads the compiled Linux kernel image (vmlinuz) and initial RAM disk image (initramfs) into main memory.
3. **Kernel Initialization Phase:** Kernel uncompresses itself, mounts `initramfs` to load essential storage drivers, initializes hardware subsystems, and mounts the real root filesystem (`/`).
4. **Init / Systemd Phase (PID 1):** Kernel spawns the first user-space process `systemd` (PID 1). `systemd` loads default target units (e.g., `multi-user.target` or `graphical.target`), initializes system daemons, and spawns the login manager.

Source: Silberschatz et al., Ch 2, p. 88-95

Unit 2: Process & Thread Management, CPU Scheduling (CO301.2)

Topic 2.1: Process Concept, Five-State Process Model & Process Control Block (PCB)

GROUP:

Consolidated Questions Covered:

- **Q1:** Draw and explain the five state process diagram.
- **Q5:** Explain with suitable diagram how fields in PCB are used to maintain process information.
- **Q7:** Draw and explain process state diagram.

1. **Five-State Process Model:** A **Process** is a program in execution. During its active lifetime, a process transitions through five distinct states:

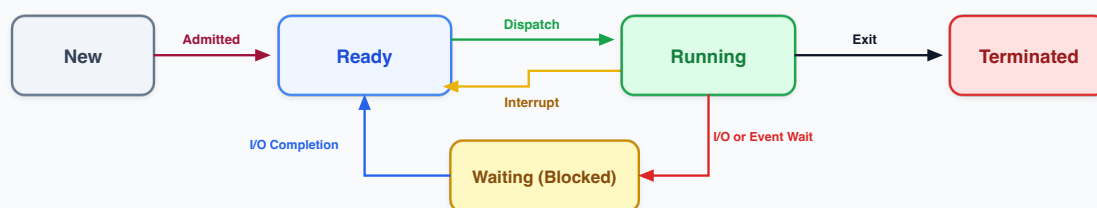


Figure 2.1: Five-State Process Lifecycle State Transitions

- **New:** Process is being created.
- **Ready:** Process is in main memory waiting to be allocated CPU time by the scheduler.
- **Running:** Instructions are actively executing on the CPU core.
- **Waiting (Blocked):** Process is waiting for an I/O event, semaphore, or signal.
- **Terminated:** Process has finished execution; OS reclaims resources.

2. Process Control Block (PCB) Structure & Fields: The OS maintains a dedicated data structure called the **PCB** (Task Struct in Linux) for every active process:

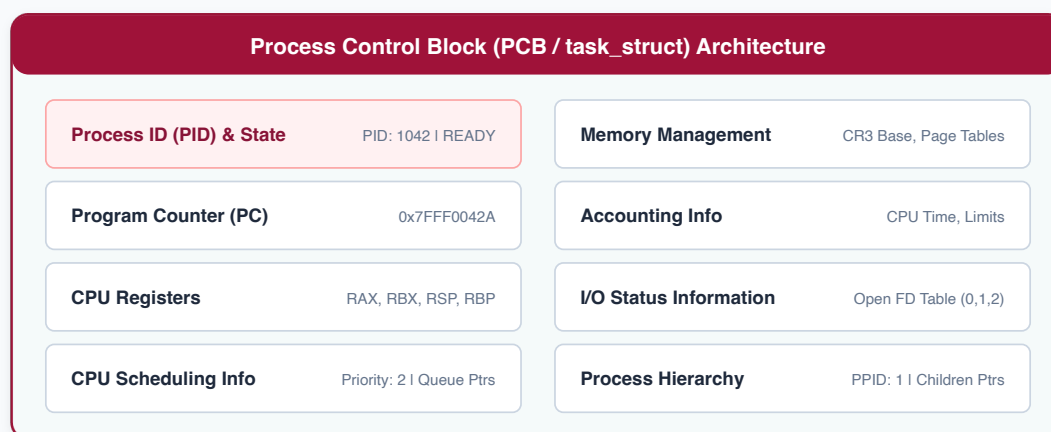


Figure 2.2: Process Control Block (PCB) Architecture

Source: Silberschatz et al., Ch 3, p. 105-115

Topic 2.2: Context Switching Mechanism & System Performance Impact

GROUP:

Consolidated Questions Covered:

- **Q4:** Analyze the context-switching mechanism and its impact on process execution and system performance.
- **Q11:** Analyze context-switching mechanism and its impact on performance.

1. Context Switching Mechanism: When a hardware interrupt or timer preemption occurs, the OS must suspend the running process P_0 and load another ready process P_1 .

1. Save the CPU context (Program Counter, CPU registers, stack pointers) of P_0 into its PCB.

2. Update P_0 's state to Ready or Blocked.
3. Move P_0 's PCB to the appropriate scheduling queue.
4. Execute scheduling algorithm to select P_1 .
5. Load P_1 's context from its PCB into the CPU hardware registers and update memory management translation tables (CR3 register / TLB flush).
6. Resume execution of P_1 at its saved Program Counter.

2. Performance Impact & Overhead:

- **Pure Computational Overhead:** Context switch time is pure waste (typically 1 to 10 microseconds) as the CPU does no useful application work during switching.
- **Cache & TLB Pollution:** Switching processes invalidates CPU L1/L2 caches and requires flushing the Translation Lookaside Buffer (TLB), leading to severe memory access latency immediately following the switch.

Source: Silberschatz et al., Ch 3, p. 115-120

Topic 2.3: Processes vs. Threads, Thread Types & Thread Lifecycle

GROUP:

Consolidated Questions Covered:

- **Q2:** Analyze differences between User-Level Threads (ULT) and Kernel-Level Threads (KLT).
- **Q3:** What is the difference between process and thread?
- **Q12:** Explain thread life cycle with state diagram.
- **Q14:** List the types of threads and differentiate them.

1. Parameter-Based Comparison: Process vs. Thread:

Parameter	Process (Heavyweight)	Thread (Lightweight)
Definition	Independent program execution unit with isolated address space	Smallest unit of CPU dispatch within a process
Address Space	Each process has its own private virtual memory space	Threads share the code, data, and heap of parent process
Creation & Context Switch	High overhead (requires creating page tables, PCB, file tables)	Very low overhead (shares memory; only registers & stack saved)
Inter-Communication	Requires formal IPC mechanisms (Pipes, Shared Memory, Sockets)	Fast direct communication via shared variables in heap
Isolation / Robustness	Crash in one process is isolated from others	A crash in one thread can terminate the entire parent process

2. User-Level Threads (ULT) vs. Kernel-Level Threads (KLT):

Parameter	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Management	Managed entirely by user-space thread library without kernel awareness	Managed directly by the Operating System Kernel
Speed	Fast creation and switching (no kernel mode switch)	Slower creation and switching (requires trap to kernel)
Blocking System Calls	If one thread blocks, the entire process blocks	If one thread blocks, kernel schedules another thread

Parameter	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Multicore Support	Cannot utilize multiple CPU cores concurrently	Kernel can schedule threads onto different CPU cores concurrently

Source: Silberschatz et al., Ch 4, p. 160-180

Topic 2.4: POSIX Pthread Implementation in C

GROUP:

Consolidated Questions Covered:

- **Q6:** Write and explain a basic pthread creation program in C.

Complete C Program Utilizing POSIX Pthreads (pthread_create & pthread_join):

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Worker thread function
void* compute_sum(void* arg) {
    int limit = *(int*)arg;
    int sum = 0;
    for (int i = 1; i <= limit; i++) {
        sum += i;
    }
    printf("[Worker Thread] Sum of 1 to %d = %d\n", limit, sum);
    pthread_exit(NULL);
}

int main() {
    pthread_t thread_id;
    int limit = 10;

    printf("[Main Process] Spawning POSIX worker thread...\n");
    // pthread_create(thread_id, attributes, function_ptr, arguments)
    if (pthread_create(&thread_id, NULL, compute_sum, &limit) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    // Main thread waits for worker thread to terminate
    pthread_join(thread_id, NULL);
    printf("[Main Process] Worker thread finished execution. Exiting.\n");
    return 0;
}
```

• Key API Primitives:

- pthread_create(): Allocates thread control block and begins execution of target function.
- pthread_join(): Suspends calling thread until target thread terminates (synchronization).
- pthread_exit(): Terminates thread and returns exit status.

Source: Silberschatz et al., Ch 4, p. 165-175

Topic 2.5: CPU Scheduling Principles (Preemptive vs. Non-Preemptive, Priority Scheduling)

GROUP:

Consolidated Questions Covered:

- **Q8:** Compare preemptive and non-preemptive scheduling with examples.
- **Q13:** Describe priority scheduling with example.

1. Parameter-Based Comparison: Preemptive vs. Non-Preemptive Scheduling:

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
CPU Allocation	CPU can be forcibly taken away from a running process by the OS	Once allocated CPU, process holds it until completion or I/O wait
Overhead	Higher overhead due to frequent context switching	Low overhead; switches only upon process termination or wait
Responsiveness	High; critical for interactive time-sharing systems	Low; short jobs can get starved behind long CPU-bound jobs
Representative Algorithms	Round Robin (RR), SRTF, Preemptive Priority	First-Come First-Served (FCFS), Non-Preemptive SJF

2. Priority Scheduling & Starvation Problem:

- **Principle:** A priority number is associated with each process; the CPU is allocated to the process with the highest priority.
- **Starvation & Aging Solution:** Low-priority processes may wait indefinitely (starvation). Solved via **Aging**—gradually increasing the priority of processes that wait in the system for a long time.

Source: Silberschatz et al., Ch 5, p. 200-220

Topic 2.6: Numerical Solved Examples — SJF & Preemptive Priority Scheduling

GROUP:

Consolidated Questions Covered:

- **Q9:** Numerical: Apply Non-Preemptive Shortest Job First (SJF). Calculate average waiting time and turnaround time with Gantt Chart.
- **Q10:** Numerical: Apply Preemptive Priority Scheduling (Lower number = Higher priority). Calculate average waiting time and turnaround time with Gantt Chart.

INTUITION:

Problem-Solving Formulas:

- **Completion Time (CT):** Time instant when process finishes execution.
- **Turnaround Time (TAT):** Total time spent in system: $TAT = CT - AT$.
- **Waiting Time (WT):** Total time spent waiting in ready queue: $WT = TAT - BT$.

Problem 1 (Q9): Non-Preemptive Shortest Job First (SJF)

Given Process Dataset:

Process	Arrival Time (AT)	Burst Time (BT)
P_1	0	3
P_2	2	6
P_3	4	4
P_4	5	2

Step-by-Step Execution Trace:

- **At $t = 0$:** Only P_1 has arrived (AT = 0, BT = 3). P_1 executes non-preemptively from $t = 0$ to $t = 3$.
- **At $t = 3$:** P_2 has arrived (AT = 2, BT = 6). P_3 and P_4 have not yet arrived. P_2 starts execution and runs until $t = 3 + 6 = 9$.
- **During P_2 's execution:** P_3 arrives at $t = 4$ (BT = 4), P_4 arrives at $t = 5$ (BT = 2).
- **At $t = 9$:** Ready processes are P_3 (BT = 4) and P_4 (BT = 2). Shortest job is P_4 . P_4 executes from $t = 9$ to $t = 11$.
- **At $t = 11$:** Only P_3 remains. P_3 executes from $t = 11$ to $t = 15$.

Non-Preemptive Shortest Job First (SJF) Gantt Chart**Figure 2.3: High-Resolution Bitmap Gantt Chart (Non-Preemptive SJF)****Result Calculation Table:**

Process	AT	BT	CT	TAT (CT - AT)	WT (TAT - BT)
P_1	0	3	3	$3 - 0 = 3$	$3 - 3 = 0$
P_2	2	6	9	$9 - 2 = 7$	$7 - 6 = 1$
P_3	4	4	15	$15 - 4 = 11$	$11 - 4 = 7$
P_4	5	2	11	$11 - 5 = 6$	$6 - 2 = 4$

Final Mathematical Averages:

- **Average Turnaround Time (Avg TAT):**

$$\text{Avg TAT} = \frac{3 + 7 + 11 + 6}{4} = \frac{27}{4} = 6.75 \text{ ms}$$

- **Average Waiting Time (Avg WT):**

$$\text{Avg WT} = \frac{0 + 1 + 7 + 4}{4} = \frac{12}{4} = 3.00 \text{ ms}$$

Problem 2 (Q10): Preemptive Priority Scheduling

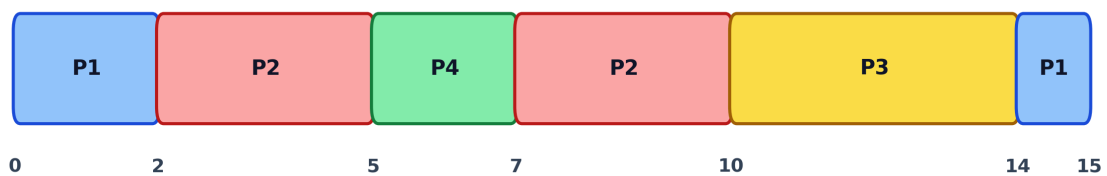
Given Process Dataset (Lower number = Higher Priority; Priority 1 is highest):

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P_1	0	3	4

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P_2	2	6	2
P_3	4	4	3
P_4	5	2	1 (Highest)

Step-by-Step Preemptive Execution Trace:

- At $t = 0$: P_1 arrives (Priority 4). Runs from $t = 0$ to $t = 2$. Remaining BT = 1.
- At $t = 2$: P_2 arrives (Priority 2). Since Priority 2 is higher than Priority 4, P_2 preempts P_1 . P_2 runs.
- At $t = 4$: P_3 arrives (Priority 3). P_2 (Priority 2) has higher priority, so P_2 continues.
- At $t = 5$: P_4 arrives (Priority 1). Since Priority 1 is higher than Priority 2, P_4 preempts P_2 ! (P_2 ran for 3 ms from $t = 2$ to 5, remaining BT = 3).
- From $t = 5$ to 7: P_4 executes for 2 ms and completes at $t = 7$.
- At $t = 7$: Ready processes are P_2 (remaining BT = 3, Pri 2), P_3 (BT = 4, Pri 3), P_1 (remaining BT = 1, Pri 4). Highest is P_2 . P_2 runs from $t = 7$ to $t = 10$ and completes.
- At $t = 10$: Ready are P_3 (Pri 3) and P_1 (Pri 4). P_3 runs from $t = 10$ to $t = 14$ and completes.
- At $t = 14$: P_1 runs remaining 1 ms from $t = 14$ to $t = 15$ and completes.

Preemptive Priority Scheduling Gantt Chart (Lower Number = Higher Priority)**Figure 2.4: High-Resolution Bitmap Gantt Chart (Preemptive Priority)****Result Calculation Table:**

Process	AT	BT	Pri	CT	TAT (CT - AT)	WT (TAT - BT)
P_1	0	3	4	15	$15 - 0 = 15$	$15 - 3 = 12$
P_2	2	6	2	10	$10 - 2 = 8$	$8 - 6 = 2$
P_3	4	4	3	14	$14 - 4 = 10$	$10 - 4 = 6$
P_4	5	2	1	7	$7 - 5 = 2$	$2 - 2 = 0$

Final Mathematical Averages:

- **Average Turnaround Time (Avg TAT):**

$$\text{Avg TAT} = \frac{15 + 8 + 10 + 2}{4} = \frac{35}{4} = 8.75 \text{ ms}$$

- **Average Waiting Time (Avg WT):**

$$\text{Avg WT} = \frac{12 + 2 + 6 + 0}{4} = \frac{20}{4} = 5.00 \text{ ms}$$

Source: Silberschatz et al., Ch 5, p. 205-230